

Documentação Técnica de Arquitetura & Sistema

MSCA Sistema de Gestão

Especificação detalhada de stack tecnológica, modelo de dados relacional, controle de concorrência, políticas de segurança RBAC e RLS, e esteiras de automação de backup em nuvem.

VERSÃO	DATA DE EMISSÃO	STATUS	REPOSITÓRIO
2.4.0 (Enterprise)	28/09/2026	Homologado em Produção	GitHub / Vercel

1. Visão Geral da Arquitetura de Software

O **MSCA Sistema de Gestão** adota o padrão de arquitetura moderna *JAMstack & Serverless BaaS* (Backend-as-a-Service), projetado para alta escalabilidade horizontal, latência reduzida no front-end e segurança corporativa em múltiplas camadas.

Front-End SPA

REACT 19

Desenvolvido em TypeScript estrito, estruturado com componentes desacoplados, roteamento via React Router DOM v7 e bundle ultra-otimizado através do Vite v8.

Backend & Storage

POSTGRESQL 15

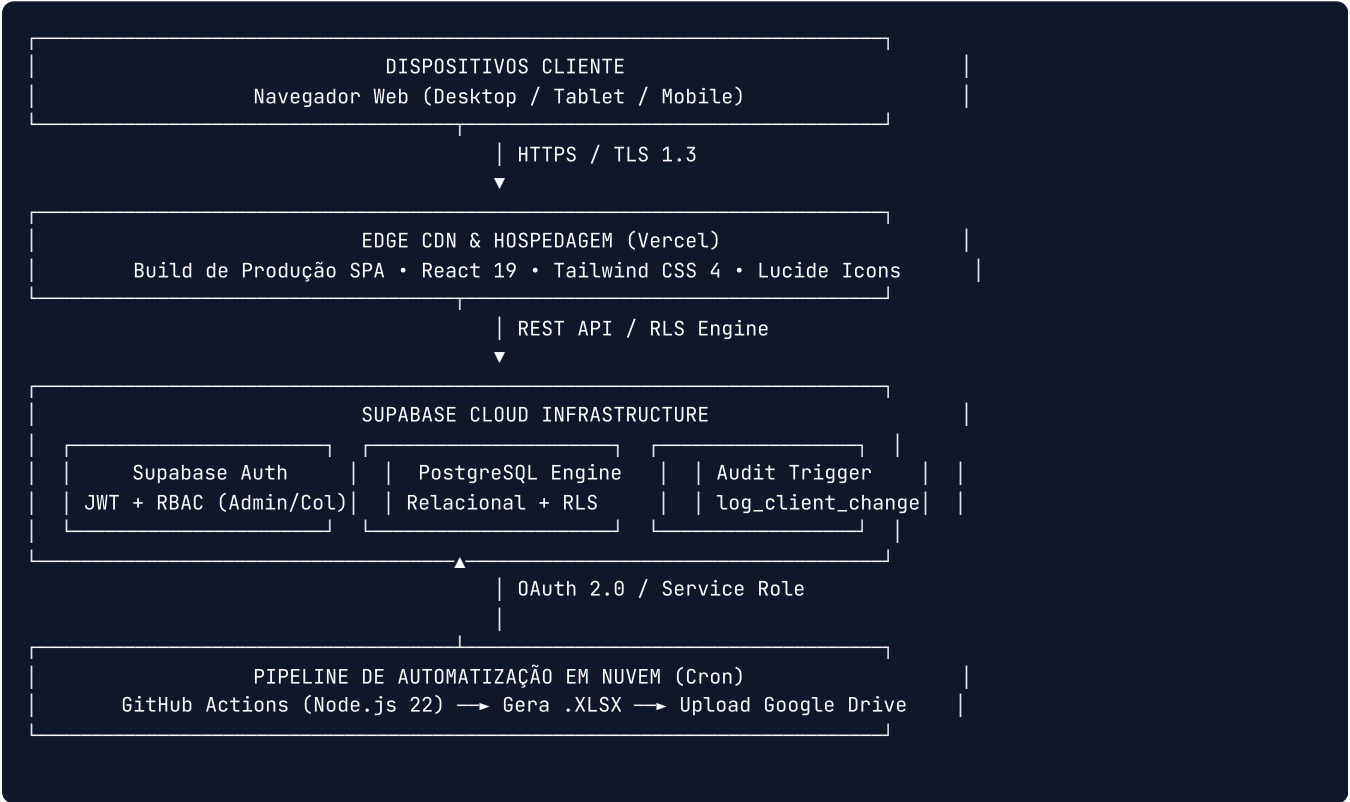
Hospedado na infraestrutura Supabase AWS, provendo banco de dados PostgreSQL com Foreign Keys, triggers transacionais e Storage integrado.

Automação Cloud

GITHUB ACTIONS

Rotinas agendadas (Cron) para backups diários automatizados e exportação para o Google Drive sem dependência de máquinas locais.

Diagrama de Comunicação do Ecossistema



2. Stack Tecnológica e Bibliotecas Principais

Tecnologia / Camada	Versão	Finalidade & Racional de Decisão
React & React DOM	19.2.8	Core da interface reativa. Renderização de alta performance, Context API para gerenciamento de estado global de sessão e toasts.
TypeScript	6.0.2	Tipagem estática estrita em 100% dos módulos, prevenindo erros em runtime e garantindo contratos de dados consistentes.
Vite & Rolldown	8.3.1	Empacotador moderno com Hot Module Replacement instantâneo e compilação de produção com tree-shaking avançado.
Tailwind CSS	4.3.3	Design system utilitário de última geração, garantindo responsividade mobile-first, paleta corporativa MSCA e zero overhead de CSS.
Supabase JS SDK	2.117.2	Cliente unificado para autenticação JWT segura, consultas tipadas ao banco de dados e execução de mutations.
ExcelJS & SheetJS	4.4.0 / 0.18.5	Geração dinâmica e estilizada de planilhas complexas com abas múltiplas, congelamento de painéis, formatação monetária e de datas.
Google APIs (googleapis)	182.0.0	Integração serverless OAuth2/Service Account para upload automatizado de artefatos de backup diretamente para o Google Drive.

3. Modelagem de Dados Relacional (Schema PostgreSQL)

O banco de dados foi estruturado com integridade referencial estrita, índices de busca em colunas de alta cardinalidade e restrições de unicidade.

Tabela	Descrição	Campos Críticos & Chaves
clients	Cadastro central de clientes contábeis (empresas e pessoas físicas).	id (PK, UUID), razao_social (NOT NULL), cnpj (UNIQUE), cpf, status (Ativo/Inativo), regime_tributario, fator_r, codigo_acesso_simples, created_at.
client_credentials	Cofre de credenciais e senhas fiscais vinculadas ao cliente.	id (PK, UUID), client_id (FK → clients.id ON DELETE CASCADE), sistema_nome (e-CAC, Prefeitura, etc), login, senha (criptografada), created_at.
financial_entries	Lançamentos de contas a receber e honorários contábeis.	id (PK, UUID), competencia (ex: out/26), client_id (FK → clients.id), valor (NUMERIC), status (À RECEBER, RECEBIDO, PERMUTA), banco, data_recebimento.
financial_expenses	Lançamentos de saídas, obrigações e despesas operacionais.	id (PK, UUID), data_pagamento_previsao (DATE), competencia, descricao_pagamento, conta_contabil, valor, status (A pagar, Pago, Descontado), banco.
profiles	Perfis de usuário estendidos da tabela auth.users.	id (PK, UUID → auth.users.id ON DELETE CASCADE), email (NOT NULL), full_name, role (ENUM: 'admin' 'colaborador'), created_at.
audit_logs	Trilha de auditoria indelével de todas as alterações críticas.	id (PK, UUID), user_id (UUID), user_email, action (INSERT/UPDATE/DELETE), entity (CLIENT/FINANCIAL/SECURITY), changes (JSONB diff), created_at.

4. Segurança, Autenticação e Controle de Acesso (RBAC & RLS)

A segurança do sistema é implementada em duas camadas complementares: no front-end (guards de rota) e no próprio banco de dados (Row Level Security do PostgreSQL).

Role: Colaborador Operacional RESTRITO - Acesso ao módulo de Cientes e consulta de dados cadastrais. - Visualização de credenciais operacionais para rotinas contábeis. - Acesso direto à pasta de arquivos do cliente no Google Drive. X Bloqueado: Não acessa módulo Financeiro nem Auditoria.	Role: Administrador Geral ACESSO TOTAL - Gestão completa de clientes, credenciais e usuários. - Acesso exclusivo ao Módulo Financeiro (Entradas, Saídas, DRE, Agenda). - Acesso exclusivo à Auditoria Completa de logs e rastreamento de IP. - Disparo de rotinas em lote, projeções recorrentes e relatórios analíticos.
---	--

Políticas de Row Level Security (RLS): Mesmo que um usuário mal-intencionado tente enviar requisições diretas via API ou Postman, as tabelas financeiras (`financial_entries` e `financial_expenses`) e de logs (`audit_logs`) rejeitam qualquer operação que não pertença a um usuário autenticado com `role = 'admin'` .

5. Motor de Projeções Recorrentes e Processamento em Lote

O sistema possui um algoritmo proprietário de geração de recebíveis futuros em lote, permitindo projetar até 12 meses de contratos de clientes ativos com prevenção contra duplicidade:

Etapa	Comportamento Técnico
1. Varredura de Base	Filtra todos os clientes com <code>status = 'Ativo'</code> que possuam valor de honorário contratual cadastrado.
2. Sequenciamento Temporal	Gera a cadeia ordenada de competências utilizando <code>generateCompetenciaSequence(initialComp, count)</code> , tratando viradas de ano automaticamente.
3. Detecção de Conflitos	Verifica a existência prévia de lançamentos para a tupla (<code>client_id</code> , <code>competencia</code>) evitando geração duplicada.
4. Inserção Transacional	Persiste os novos lançamentos com status inicial <code>À RECEBER</code> e vinculação automática com a conta bancária padrão.

6. Pipeline de Backup Diário em Nuvem (GitHub Actions)

A rotina de backup é 100% autônoma, operando sem necessidade de servidores dedicados ou computadores ligados no escritório.

Especificação do Workflow (

`.github/workflows/daily_backup.yml`

)

- **Gatilho Cron:** Disparado diariamente às `03:00 UTC` (equivalente a `00:00 Horário de Brasília`).
- **Gatilho Manual:** Suporte a `workflow_dispatch` para acionamento sob demanda pelo painel do GitHub.
- **Ambiente:** Runner Linux Ubuntu Latest com runtime Node.js v22.
- **Segurança:** Todas as chaves do Supabase e tokens OAuth2 do Google Drive são injetados em tempo de execução via *GitHub Actions Encrypted Secrets*.

Algoritmo de Extração com Paginação

```
async function fetchAll(table, selectQuery = '*', orderCol = 'created_at', ascending = false) {
  let allRows = [];
  let from = 0;
  const PAGE_SIZE = 1000;
  let hasMore = true;

  while (hasMore) {
    const { data, error } = await supabase
      .from(table)
      .select(selectQuery)
      .order(orderCol, { ascending })
      .range(from, from + PAGE_SIZE - 1);

    if (error) break;
    if (data && data.length > 0) allRows = allRows.concat(data);
    if (!data || data.length < PAGE_SIZE) hasMore = false;
    else from += PAGE_SIZE;
  }
  return allRows;
}
```

7. Trilha de Auditoria e Conformidade

Em conformidade com boas práticas de governança e segurança da informação, todas as ações sensíveis realizadas no sistema disparam registros imutáveis:

Eventos Rastreados

- Criação, edição e exclusão de clientes.
- Criação, edição e exclusão de credenciais de acesso.
- Liquidação, baixa em lote ou exclusão de lançamentos financeiros.
- Tentativas de acesso não autorizado e troca de senhas.

Metadados Registrados

- ID e e-mail do operador responsável.
- Endereço IP e User-Agent da requisição.
- Timestamp exato (UTC e horário de Brasília).
- Diff estruturado em JSON comparando os valores anteriores e atuais.